

1. Basic Commands of Linux OS: Navigation (ls, cd), File manipulation (cp, mv, rm), and Viewing (cat, less), where is, what is and their options

Objective

To build fundamental command-line interface (CLI) fluency in a Linux environment, enabling students to navigate the filesystem hierarchy, perform file management operations safely, inspect textual data, and retrieve command locations and manual documentation.

Learning Outcomes

By the end of this module/lab, students will be able to:

- **Navigate** the Linux directory tree using absolute and relative paths with cd and ls.
- **Execute** file management tasks (copying, moving, renaming, and removing) using cp, mv, and rm.
- **Inspect** text files efficiently using whole-file output (cat) and paginated viewing (less).
- **Locate** system binaries, source code, and documentation using whereis and whatis.
- **Utilize** standard command options (-l, -a, -r, -i, -f) to control command execution and prevent accidental data loss.

Hands-On Lab Procedure

1.1. Directory Creation & Navigation Setup: Environment Readiness.

Open a bash terminal and construct a temporary workspace inside your home directory.

Check current directory location

```
pwd
```

- Print current/working directories

Create a project directory and move into it

```
mkdir -p ~/linux_lab/docs
```

- mkdir → **make directory**
- -p → creates **parent directories automatically** if they don't already exist.
- ~ → your **home directory**
- linux_lab → a directory called linux_lab
- docs → a directory inside linux_lab

```
cd ~/linux_lab
```

- **cd** → **change directory**
- `~/linux_lab` → tells Linux to enter the `linux_lab` directory inside your home directory.

2.2. File Creation & Manipulation:

Content Generation.

Generate sample text files, copy them across directories, and test safe file operations.

Bash

Create sample files using redirection

```
echo "Line 1: Linux system administration" > sample.txt
```

```
echo "Line 2: Command line interface basics" >> sample.txt
```

Copy sample.txt into the docs subdirectory

```
cp -v sample.txt docs/sample_backup.txt
```

Rename sample.txt to primary.txt using mv

```
mv -v sample.txt primary.txt
```

3.3. Viewing File Contents: File Inspection.

Examine file contents using both immediate print and paginated view modes.

Bash

View with line numbers

```
cat -n primary.txt
```

View system files with pagination (Press 'q' to exit less)

```
less /etc/passwd
```

4.4. Binary & Documentation Inquiry: System Querying.

Use system utility commands to discover executable paths and short command summaries.

Locate command binary and manual page paths

```
whereis ls
```

Print single-line manual descriptions

```
whatis cat
```

```
whatis less
```

Note: If “**whatis**” command doesn’t work on the system. Run **sudo mandb** in the terminal to update the command database.

5.5. Safe Directory Cleanup: Environment Cleanup.

Prompt for interactive confirmation before removing created test files.

```
# Move back to home directory
```

```
cd ~
```

```
# Interactively remove the lab directory and contents
```

```
rm -ri ~/linux_lab
```

Detailed Command & Options Reference

1. Navigation Commands

ls (List Directory Contents)

Lists files and directories in the target path.

Bash

```
ls [OPTIONS] [DIRECTORY]
```

- **-l**: Uses long listing format (displays permissions, owner, group, file size, and modification date).
- **-a**: Shows all files, including hidden files (starting with .).
- **-h**: Prints sizes in human-readable format (e.g., 1K, 234M, 2G) when used with **-l**.
- **-R**: Lists subdirectories recursively/reversely.
- **-t**: Sorts entries by modification time (newest first).

Example: `ls -lah /var/log`

cd (Change Directory)

Changes the current working directory in the shell environment.

Bash

```
cd [DIRECTORY_PATH]
```

- **cd ~** or **cd**: Changes to the user's home directory.
- **cd ..**: Moves up one level to the parent directory.
- **cd -**: Switches back to the previous working directory.
- **cd /**: Moves to the root directory of the filesystem.

2. File Manipulation Commands

cp (Copy Files and Directories)

Copies files or directories from a source location to a destination.

Bash

```
cp [OPTIONS] SOURCE DESTINATION
```

- **-r / -R**: Copies directories recursively (required when copying folders). Copy all content of source folder to destination folder.
- **-i**: Interactive mode; prompts before overwriting existing destination files.
- **-v**: Verbose mode; explains what is being done.
- **-u**: Updates; copies only when the source file is newer than the destination file.

Example: cp -ri my_folder/ /backup/my_folder/

Quick Revision Table

Command	Purpose	Example
cp	Copy file	cp a.txt b.txt
cp -r	Copy directory and contents	cp -r dir1 dir2
cp -i	Ask before overwriting	cp -i a.txt b.txt
cp -v	Show what is being copied	cp -v a.txt b.txt
cp -u	Copy only if source is newer	cp -u a.txt b.txt

mv (Move / Rename Files)

Moves files or directories to a new location, or renames them if the path remains unchanged.

Bash

```
mv [OPTIONS] SOURCE DESTINATION
```

- **-i**: Prompts before overwriting an existing destination file.
- **-f**: Force mode; overwrites target files without prompting.
- **-v**: Verbose mode; prints source and destination details.
- **-n**: Do not overwrite any existing file.

Example: `mv -v old_name.txt new_name.txt`

mv — Quick Revision Table

Option	Meaning	Quick Example
<code>mv SOURCE DESTINATION</code>	Move or rename	<code>mv a.txt docs/</code>
<code>-i</code>	Ask before overwriting	<code>mv -i a.txt docs/</code>
<code>-f</code>	Force overwrite, no prompt	<code>mv -f a.txt docs/</code>
<code>-v</code>	Show what is being moved	<code>mv -v a.txt docs/</code>
<code>-n</code>	Never overwrite existing file	<code>mv -n a.txt docs/</code>

rm (Remove Files / Directories)

Deletes specified files or directories from the filesystem.

Bash

```
rm [OPTIONS] FILE...
```

- **-r / -R**: Removes directories and their contents recursively.
- **-f**: Force mode; ignores non-existent files and overrides warnings without prompting.
- **-i**: Prompts for confirmation before every removal.
- **-d**: Removes empty directories.

Warning: Commands like `rm -rf` delete data permanently without passing through a trash folder.

Quick Revision

Command	Meaning
<code>rm file.txt</code>	Delete a file
<code>rm -r folder</code>	Delete folder and its contents
<code>rm -f file.txt</code>	Force delete
<code>rm -i file.txt</code>	Ask before deleting
<code>rm -d folder</code>	Delete an empty directory
<code>rm -rf folder</code>	Forcefully delete folder and contents ⚠

3. Viewing Commands

cat (Concatenate and Display)

Reads one or more files and prints their full content sequentially to standard output. Best suited for short files.

Bash

```
cat [OPTIONS] FILE...
```

- **-n**: Numbers all output lines.
- **-b**: Numbers non-empty output lines only.
- **-E**: Displays a \$ sign at the end of each line.
- **-s**: Suppresses repeated empty output lines into a single blank line.

Example: cat -n config.conf

Quick Revision Table

Command	What it does
cat file.txt	Displays file contents
cat -n file.txt	Numbers all lines
cat -b file.txt	Numbers non-empty lines
cat -E file.txt	Shows \$ at end of each line
cat -s file.txt	Removes repeated blank lines from output
cat file1 file2	Displays files sequentially

less (Paginated Viewer)

Opens a text file in an interactive interface allowing backward and forward navigation without loading the whole file into memory.

The **less command** is used to **view a text file one screen at a time**. It is especially useful for **large files** because you don't have to display the entire file on the terminal at once.

Bash

```
less [OPTIONS] FILE
```

- **-N**: Shows line numbers within the viewing interface.

- **/pattern**: Searches forward for the specified pattern (press n for next match, N for previous).
- **-G**: Jumps to the end of the file.
- **-g**: Jumps to the beginning of the file.
- **q**: Exits the less viewer interface.

4. Information & Help Commands

whereis

Locates the binary executable, source code files, and manual page files for a specified command. Alternatively, the **whereis command** is used to find the **location of a Linux command's executable (binary), source code, and manual page**.

Bash

```
whereis [OPTIONS] COMMAND_NAME
```

- **-b**: Searches only for binary executables.
- **-m**: Searches only for manual page documentation.
- **-s**: Searches only for source code entries.

Example: whereis -b gcc

whatis

Searches the index database of man pages to display a concise, single-line description of a command. It means the **whatis command** is used to get a **short, one-line description of a Linux command**.

Bash

```
whatis [OPTIONS] COMMAND_NAME
```

- **-w**: Enables wildcard matching in searches.
- **-r**: Uses regular expression search matching.

Example: whatis cp mv rm

"Imagine you're organizing your computer. First, let's create a folder."

Then demonstrate:

`mkdir students`

`cd students`

`touch notes.txt`

`ls`

create folder → enter folder → create file → see file

Then build the next concept on top of it:

`mkdir backup`

`cp notes.txt backup/`

`mv notes.txt linux-notes.txt`

`rm linux-notes.txt`

For Beginners

- **Where am I?** → `pwd`
- **Who am I?** → `whoami`
- **What's here?** → `ls`
- **How do I move?** → `cd`
- **How do I make a folder?** → `mkdir`
- **How do I make a file?** → `touch`, `echo`
- **How do I copy something?** → `cp`
- **How do I rename/move something?** → `mv`
- **How do I delete something?** → `rm`
- **How do I read a file?** → `cat`
- **Who can access this file?** → `ls -l`
- **What is the meaning of this command?** → `whatis`
- **Where is the details of this command?** → `whereis`